# Approach Document

## 1. Architecture

The system is a conversational retrieval agent with a strict separation between retrieval (deterministic) and generation (LLM-based).

```
[User POST /chat]
   └──> [State Extractor] (Builds JSON from history)
           └──> [Intent Detection] (Recommend / Clarify / Refuse)
                   ├──> (If Refuse) ──────> [Refusal Response]
                   └──> (If Recommend) ──> [BM25 Search] ──> [Metadata Scoring] ──>
```

## 2. Retrieval Design

**Why hybrid retrieval maximises Recall@10:**

Stage 1 — BM25: Okapi BM25 with k1 = 1.5, b = 0.75 over a composite document field (name + description + competencies + technical_domains + use_cases). Catches exact keyword signals like "Java", "Docker", "HIPAA", "DSI".

Stage 2 — Metadata Scoring: Five orthogonal signals combined with learned weights:

| Signal | Weight | Rationale |
|---|---|---|
| BM25 (normalised) | 0.35 | Keyword recall anchor |
| Competency overlap | 0.25 | Role-skill alignment |
| Test type preference | 0.15 | User expressed needs |
| Seniority match | 0.10 | Avoids level mismatch |
| Use case match | 0.10 | Selection vs development |
| Language match | 0.05 | Constraint satisfaction |

Stage 3 — LLM reranking: Groq Llama 3.3 70b receives the top-10 pre-retrieved candidates and conversation context. LLM selects the optimal 2-8 items, writes rationale, and produces the table.

Stage 4 — Validation: Every URL in the LLM output is checked against the catalog. Any hallucinated item is dropped. Fallback to retrieval results if LLM produces no valid items.

**Why not pure embedding/vector search?** For a catalog of ~60 assessments, BM25 outperforms embedding models on exact-match recall for technical skills ("Docker", "HIPAA", "SVAR"). Embeddings add noise for short domain-specific terms. Our multi-signal scoring provides the semantic layer without embedding overhead.

## 3. State Reconstruction

Every API call receives the full conversation history. A deterministic extractor builds a structured state object:

```
{
  "role": "java_developer", "seniority": "professional",
  "technical_skills": ["java", "spring"], "personality_focus": false,
```

```
  "language_requirements": ["english_us"], "use_case": "selection",
  "turn_count": 4
}
```

This state is the single source of truth. No memory = no state bugs.

## 4. Clarification Policy

**Turn budget: 8 turns.**

Rules:

1. Never ask more than one clarification round (compound questions only)
2. If turn_count >= 6, always recommend
3. Ask only when role AND seniority are both unknown

**Bad pattern (wastes turns):**

> "What role is this for?" → User answers → "What seniority?" → User answers → "Personality or cognitive?" → User answers → (turn 4 before first recommendation)

**Our pattern:**

> "Could you tell me: the role title and seniority level / whether personality or cognitive assessments are important?" → User answers → Recommend immediately

## 5. Evaluation Methodology

Automated test suite (40+ tests):

| Category | Count | Pass Target |
|---|---|---|
| Catalog integrity | 10 | 100% |
| State extraction | 10 | 100% |
| Retrieval recall | 10 | ≥70% |
| Refusal detection | 6 | 100% |
| Schema compliance | 4 | 100% |
| Hallucination prevention | 4 | 100% |
| Behavior probes | 5 | ≥80% |

**Recall@10 computation:**

```
recall = |retrieved_top_10 ∩ ground_truth| / |ground_truth|
```

Target: ≥0.7 across all 10 sample conversation scenarios.

## 6. Tradeoffs & Failed Experiments

**Tried: Pure semantic embedding search** — Lower recall for exact technical terms (Java, Docker). Dropped.

**Tried: Single large prompt with full catalog** — 60 items × average 500 tokens = 30k tokens per call, slow and expensive. Switched to pre-retrieval → LLM reranking.

**Tried: Per-question clarification** — Burns turns. Switched to compound single-question policy.

**Tradeoff: No persistent vector store** — For a 60-item catalog, BM25 in-memory is sufficient and instant. At 1000+ items, would add FAISS or pgvector.

**Tradeoff: Groq Llama 3.3 70b** — High-speed API inference, large 128k context window, excellent markdown table instruction compliance.